

Zaawansowane Technologie Baz Danych

Porównanie wydajności systemów zarządzania bazami danych:
PostgreSQL, MySQL, MongoDB, Redis

Sprawozdanie projektowe

Politechnika Krakowska
Kierunek: Informatyka

Kwiecień 2026

Spis treści

1	Cel i zakres pracy	3
1.1	Hipoteza badawcza	3
2	Opis wybranych systemów zarządzania bazami danych	3
2.1	PostgreSQL 16	3
2.2	MySQL 8.0	3
2.3	MongoDB 7	4
2.4	Redis 7	4
3	Awaryjność, bezpieczeństwo, migracje, integracje i skalowalność	5
4	Opis zbioru danych	5
4.1	Modele danych w systemach nierzelacyjnych	6
4.2	Skale danych testowych	7
5	Opis aplikacji testowej	7
5.1	Wymagania	7
5.2	Wykorzystane technologie i narzędzia	7
5.3	Struktura projektu i opis działania	7
6	Opis scenariuszy testowych	8
7	Wyniki testów wydajnościowych	10
7.1	Wyniki bez indeksów	10
7.2	Wyniki z indeksami	10
7.3	Porównanie: wpływ indeksów na wydajność	11
8	Wizualizacja wyników	12
9	Analiza indeksów	18
9.1	Zastosowane indeksy	18
10	Analiza planów zapytań (EXPLAIN)	19
10.1	PostgreSQL – EXPLAIN ANALYZE	19
10.2	MySQL – EXPLAIN FORMAT=JSON	19
10.3	MongoDB – explain executionStats	20
11	Weryfikacja hipotezy badawczej	20
11.1	Analiza wyników	20
11.2	Wnioski dotyczące hipotezy	20
12	Rozszerzona analiza wyników	21
12.1	Skalowalność	21
12.2	Porównanie modeli danych	21
12.3	Dane półustrukturalne (JSON)	22
12.4	Automatyzacja testów	22

13 Podsumowanie i wnioski

22

1 Cel i zakres pracy

Celem projektu jest porównanie wydajności czterech systemów zarządzania bazami danych (SZBD) przy wykonywaniu operacji CRUD (Create, Read, Update, Delete) na dużych zbiorach danych. Analizie poddano:

- **2 systemy relacyjne:** PostgreSQL 16, MySQL 8.0
- **2 systemy nierelacyjne:** MongoDB 7 (dokumentowa), Redis 7 (klucz-wartość)

Zakres pracy obejmuje:

1. Zaprojektowanie schematu danych systemu szpitalnego (13 tabel relacyjnych, model dokumentowy MongoDB, struktury Redis).
2. Implementację aplikacji testowej automatyzującej generowanie danych, wstawianie, benchmarki i analizę.
3. Przeprowadzenie 24 scenariuszy testowych CRUD (6 na każdą operację) w trzech skalach: 10 000, 100 000, 500 000 rekordów.
4. Porównanie wydajności przed i po zastosowaniu indeksów.
5. Analizę planów zapytań (EXPLAIN).
6. Weryfikację hipotezy badawczej o wpływie indeksów na wydajność.

1.1 Hipoteza badawcza

H1: Indeksy a wydajność – Zastosowanie indeksów znacząco poprawia wydajność operacji SELECT kosztem spadku wydajności operacji INSERT i UPDATE.

2 Opis wybranych systemów zarządzania bazami danych

2.1 PostgreSQL 16

PostgreSQL jest zaawansowanym, obiektowo-relacyjnym systemem baz danych o otwartym kodzie źródłowym. Oferuje pełną zgodność z ACID, zaawansowane mechanizmy indeksowania (B-tree, Hash, GIN, GiST, BRIN), obsługę JSON/JSONB, widoki zmateriaлизованne, partycjonowanie tabel oraz replikację logiczną i strumieniową.

Zalety: Zaawansowane typy danych, rozbudowany optymalizator zapytań, pełna zgodność ze standardem SQL, doskonała obsługa transakcji, MVCC (Multi-Version Concurrency Control), wsparcie dla procedur składowanych w wielu językach (PL/pgSQL, PL/Python).

Wady: Wyższe zużycie pamięci w porównaniu z MySQL, wolniejsze wstawianie danych masowych bez odpowiedniej konfiguracji, bardziej złożona konfiguracja.

Zastosowania biznesowe: Systemy finansowe, systemy ERP, analityka danych, systemy GIS (PostGIS), platformy e-commerce.

2.2 MySQL 8.0

MySQL jest najpopularniejszym relacyjnym systemem baz danych o otwartym kodzie. Wykorzystuje silnik InnoDB z obsługą transakcji ACID, mechanizmem MVCC, kluczami

obcymi i indeksami B-tree. MySQL 8.0 wprowadził okna analityczne, CTE (Common Table Expressions) i ulepszony optymalizator.

Zalety: Wysoka wydajność operacji odczytu, łatwość konfiguracji, szeroka kompatybilność, duża społeczność, niskie wymagania sprzętowe.

Wady: Ograniczone wsparcie dla zaawansowanych typów danych, brak niektórych funkcji SQL standardu (np. pełna obsługa sekwencji), historycznie słabsza optymalizacja podzapytań.

Zastosowania biznesowe: Aplikacje webowe (WordPress, Joomla), systemy CMS, platformy SaaS, systemy logistyczne.

2.3 MongoDB 7

MongoDB jest dokumentową bazą danych NoSQL, przechowującą dane w formacie BSON (Binary JSON). Oferuje elastyczny schemat, replikację zestawów replik (Replica Sets), sharding dla skalowalności horyzontalnej oraz zaawansowane indeksowanie (pojedyncze, złożone, wielokluczowe, tekstowe, geoprzestrzenne).

Zalety: Elastyczny schemat (schema-less), naturalne modelowanie danych hierarchicznych, szybkie operacje na pojedynczych dokumentach, łatwa skalowalność horyzontalna, wbudowana agregacja.

Wady: Brak transakcji wielodokumentowych (do wersji 4.0), większe zużycie pamięci dyskowej, brak JOIN (wymaga denormalizacji lub \$lookup), ewentualna spójność w konfiguracjach rozproszonych.

Zastosowania biznesowe: Systemy zarządzania treścią, aplikacje mobilne, IoT, katalogi produktów, systemy analityki czasu rzeczywistego.

2.4 Redis 7

Redis (Remote Dictionary Server) jest bazą danych typu klucz-wartość działającą w pamięci RAM. Obsługuje struktury danych: stringi, hashe, listy, zbiory, zbiory uporządkowane, strumienie. Oferuje persystencję (RDB, AOF), replikację master-slave i klaster Redis.

Zalety: Ekstremalnie niska latencja (< 1 ms), bogate struktury danych, atomowość operacji, Pub/Sub, obsługa TTL (Time-To-Live), skrypty Lua.

Wady: Ograniczona pojemność (limitowana pamięcią RAM), brak złożonych zapytań (brak SQL), brak relacji między danymi, utrata danych w razie awarii (zależy od konfiguracji persystencji).

Zastosowania biznesowe: Cache'owanie, sesje użytkowników, kolejki wiadomości, tablice wyników w grach, systemy czasu rzeczywistego, liczniki i metryki.

3 Awaryjność, bezpieczeństwo, migracje, integracje i skalowalność

Tabela 1: Porównanie cech niefunkcjonalnych analizowanych SZBD

Cecha	PostgreSQL	MySQL	MongoDB	Redis
Awaryjność	WAL, Point-in-Time Recovery, replikacja strumieniowa	InnoDB Crash Recovery, replikacja master-slave, Group Replication	Replica Sets z auto-failover, journal	RDB snapshots, AOF, Sentinel dla HA
Bezpieczeństwo	Role, uprawnienia, SSL/TLS, Row-Level Security, szyfrowanie at-rest	Role, uprawnienia, SSL/TLS, szyfrowanie InnoDB tablespace	SCRAM-SHA-256, Role-Based Access, szyfrowanie at-rest, SSL/TLS	ACL (od v6), hasła, SSL/TLS, brak szyfrowania at-rest
Migracje	pg_dump/pg_restore, replikacja logiczna, pgLoader	mysqldump, MySQL Shell, replikacja	mongodump/mongorestore, Atlas Live Migration	RDB/AOF, redis-cli -pipe, Redis Data Integration
Integracje	Foreign Data Wrappers, Kafka, Debezium, REST API	Connectors (JDBC, ODBC), MySQL Router	Connectors natywne, Atlas Triggers, Kafka Connector	Pub/Sub, Streams, integracja z Celery, Sidekiq
Skalowalność	Skalowanie pionowe, Citus dla horyzontalnego, partycjonowanie	Skalowanie pionowe, MySQL NDB Cluster, ProxySQL	Sharding natywny, skalowalność horyzontalna	Redis Cluster (do 1000 węzłów), sharding automatyczny

4 Opis zbioru danych

Dane testowe modelują system informatyczny szpitala. Schemat relacyjny składa się z **13 tabel**:

Tabela 2: Schemat relacyjny – tabele systemu szpitalnego

Tabela	Typ	Opis / klucze obce
departments	Słownikowa	Oddziały szpitalne (id, name, phone)
specializations	Słownikowa	Specjalizacje lekarskie (id, name)
diseases	Słownikowa	Choroby z kodem ICD-10 (id, icd10_code, name)
medical_services	Słownikowa	Usługi medyczne z ceną bazową
medications	Słownikowa	Leki (id, name, active_substance)
patients	Główna	Pacjenci (id, PESEL, imię, nazwisko, data ur., płeć)
doctors	Główna	Lekarze → departments, specializations
visits	Transakcyjna	Wizyty → patients, doctors
performed_services	Transakcyjna	Wykonane usługi → visits, medical_services
diagnoses	Transakcyjna	Diagnozy → visits, diseases
prescriptions	Transakcyjna	Recepty → visits
prescription_items	Transakcyjna	Pozycje recept → prescriptions, medications
test_results	Transakcyjna	Wyniki badań → visits

4.1 Modele danych w systemach nierelacyjnych

MongoDB – model dokumentowy z zagnieżdżeniami: Pacjent jest dokumentem głównym, zawierającym tablicę wizyt, a każda wizyta posiada zagnieżdżone tablice: wykonane usługi, diagnozy, recepty (z pozycjami) oraz wyniki badań. Taki model eliminuje konieczność JOINów.

Redis – model klucz-wartość: Statusy wizyt przechowywane jako stringi (`visit:status:<id>`), sesje lekarzy jako hashe (`session:doctor:<id>`). Redis wykorzystywany jako warstwa cache i szybkiego dostępu.

4.2 Skale danych testowych

Tabela 3: Liczba rekordów w zależności od skali

Tabela	10 000	100 000	500 000
departments	10	20	30
specializations	15	25	40
diseases	100	400	800
medical_services	50	150	300
medications	80	300	600
patients	2 000	15 000	50 000
doctors	50	150	400
visits	10 000	100 000	500 000
performed_services	~10 000	~100 000	~500 000
diagnoses	~10 000	~100 000	~500 000
prescriptions	~4 000	~40 000	~200 000
prescription_items	~8 000	~80 000	~400 000
test_results	~6 000	~60 000	~300 000
Łączna suma (SQL)	~50 000	~500 000	~2 500 000

5 Opis aplikacji testowej

5.1 Wymagania

- Automatyczne generowanie syntetycznych danych medycznych przy użyciu biblioteki Faker.
- Masowe wstawianie danych (Bulk Insert) do 4 systemów bazodanowych.
- Wykonanie 24 scenariuszy CRUD w trybie bez indeksów i z indeksami.
- Pomiar czasu operacji (średnia z 3 prób) z dokładnością do mikrosekundy.
- Automatyczne generowanie raportów EXPLAIN i wykresów porównawczych.

5.2 Wykorzystane technologie i narzędzia

- **Python 3.13** – język implementacji aplikacji.
- **Docker + Docker Compose** – konteneryzacja wszystkich 4 baz danych.
- **psycopg2** – sterownik PostgreSQL.
- **mysql-connector-python** – sterownik MySQL.
- **pymongo** – sterownik MongoDB.
- **redis-py** – sterownik Redis.
- **Faker** – generowanie realistycznych danych syntetycznych.
- **pandas + matplotlib** – analiza wyników i wizualizacja.
- **L^AT_EX** – skład sprawozdania.

5.3 Struktura projektu i opis działania

Listing 1: Struktura katalogów projektu

```

ZTDB/
  docker-compose.yml      # 4 kontenery: PostgreSQL, MySQL, MongoDB,
                           Redis
  run_all.py              # Główny skrypt automatyzacji
  core/
    database.py           # ConnectionManager, wzorzec Abstract Factory
    generator.py          # DataGenerator - generowanie danych Faker
    seeder.py             # DatabaseSeeder - bulk insert, indeksy,
                           schemat
    benchmark.py          # BenchmarkEngine - 24 scenariusze CRUD
  results/
    results_all.csv       # Zbiorczy plik z wynikami
    explain_report.txt    # Plany zapytan EXPLAIN
    charts/              # Wykresy PNG
  sprawozdanie/
    sprawozdanie.tex      # Niniejsze sprawozdanie

```

Aplikacja działa w pełni automatycznie. Polecenie `python run_all.py` kolejno:

1. Sprawdza połączenia z 4 bazami danych (ping).
2. Dla każdej skali (10k, 100k, 500k):
 - (a) Generuje dane w pamięci (Faker).
 - (b) Wstawia dane do PostgreSQL, MySQL, MongoDB, Redis.
 - (c) Usuwa indeksy, wykonuje benchmark (24 scenariusze $\times$ 3 próby).
 - (d) Tworzy indeksy, wykonuje benchmark ponownie.
 - (e) Generuje raporty EXPLAIN.
3. Łączy wyniki CSV i generuje wykresy porównawcze.

6 Opis scenariuszy testowych

Zdefiniowano **24 scenariusze testowe** – po 6 dla każdej operacji CRUD. Każdy scenariusz wykonywany jest 3 razy, a raportowana jest średnia arytmetyczna czasu wykonania. Poniżej zestawienie wszystkich scenariuszy:

Tabela 4: Scenariusze testowe – operacje CREATE

Nr	Nazwa	Opis
C1	insert_patient	Wstawienie nowego pacjenta
C2	insert_visit	Wstawienie nowej wizyty (z FK do pacjenta i lekarza)
C3	insert_diagnosis	Wstawienie diagnozy (z FK do wizyty i choroby)
C4	insert_prescription_with_items	Wstawienie recepty z pozycjami
C5	insert_performed_service	Wstawienie wykonanej usługi medycznej
C6	insert_test_result	Wstawienie wyniku badania laboratoryjnego

Tabela 5: Scenariusze testowe – operacje READ

Nr	Nazwa	Opis
R1	select_patient_by_id	Odczyt pacjenta po kluczu głównym
R2	select_visits_with_doctor	JOIN wizyt z lekarzami po patient_id
R3	select_visit_diagnoses	JOIN diagnoz z chorobami po visit_id
R4	select_patient_full_history	Wielokrotny JOIN: pacjent + wizyty + usługi
R5	select_aggregated_costs	Agregacja SUM kosztów usług (GROUP BY)
R6	select_prescriptions_with_meds	Trójkrotny JOIN: recepty + pozycje + leki

Tabela 6: Scenariusze testowe – operacje UPDATE

Nr	Nazwa	Opis
U1	update_patient_name	Zmiana nazwiska pacjenta po id
U2	update_visit_status	Zmiana statusu wizyty po id
U3	update_service_price	Zmiana ceny usługi po visit_id
U4	update_diagnosis_notes	Aktualizacja notatki diagnozy po visit_id
U5	update_doctor_license	Zmiana numeru licencji lekarza po id
U6	update_department_phone	Zmiana telefonu oddziału po id

Tabela 7: Scenariusze testowe – operacje DELETE

Nr	Nazwa	Opis
D1	delete_test_result	Usunięcie wyniku badania po id
D2	delete_diagnosis	Usunięcie diagnozy po id
D3	delete_patient	Usunięcie pacjenta po id
D4	delete_performed_service	Usunięcie usługi po id
D5	delete_prescription	Usunięcie recepty po id
D6	delete_visit_cascade	Usunięcie wizyty (kaskadowe z FK)

7 Wyniki testów wydajnościowych

7.1 Wyniki bez indeksów

Tabela 8: Średni czas operacji CRUD bez indeksów [ms] – uśredniony po 6 scenariuszach

Operacja	Skala	PostgreSQL	MySQL	MongoDB	Redis
CREATE	10 000	2.74	12.24	0.92	0.39
	100 000	6.85	9.19	0.89	0.43
	500 000	18.90	9.55	1.00	0.45
READ	10 000	1.07	0.90	1.00	0.41
	100 000	5.60	0.95	1.20	0.40
	500 000	16.61	1.16	1.88	0.41
UPDATE	10 000	1.05	3.45	1.42	0.38
	100 000	2.02	3.25	5.89	0.40
	500 000	7.69	3.72	22.31	0.40
DELETE	10 000	2.81	8.60	1.43	0.77
	100 000	5.95	9.90	1.45	0.77
	500 000	20.78	11.75	1.47	0.81

7.2 Wyniki z indeksami

Tabela 9: Średni czas operacji CRUD z indeksami [ms] – uśredniony po 6 scenariuszach

Operacja	Skala	PostgreSQL	MySQL	MongoDB	Redis
CREATE	10 000	2.49	10.62	1.08	0.50
	100 000	2.54	9.65	1.09	0.40
	500 000	2.58	11.79	1.21	0.43
READ	10 000	0.54	2.34	0.89	0.43
	100 000	0.61	12.87	0.91	0.37
	500 000	0.57	0.98	1.01	0.40
UPDATE	10 000	1.01	3.26	1.74	0.41
	100 000	1.16	6.09	5.23	0.39
	500 000	0.95	3.97	9.60	0.40
DELETE	10 000	2.54	9.13	1.69	0.79
	100 000	2.46	8.89	1.42	0.75
	500 000	2.48	8.77	1.44	0.78

7.3 Porównanie: wpływ indeksów na wydajność

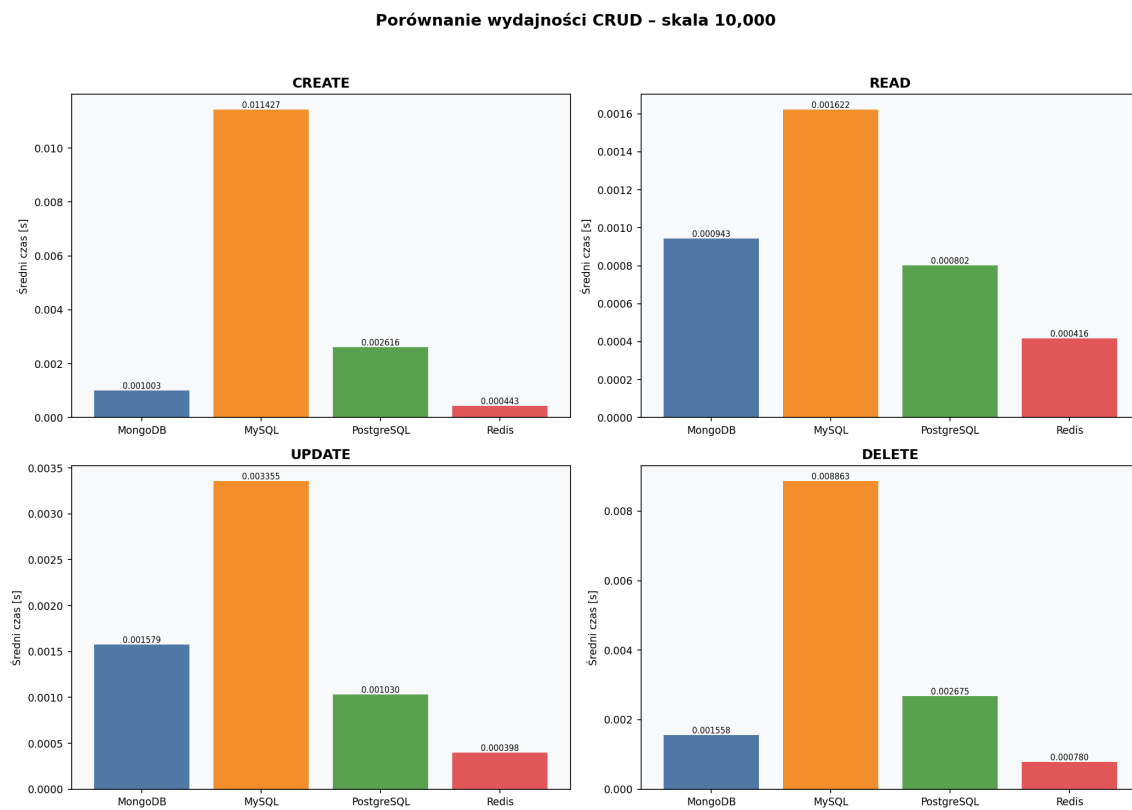
Tabela 10: Zmiana wydajności po zastosowaniu indeksów (skala 500 000) – wartości ujemne = przyspieszenie

Operacja	PostgreSQL	MySQL	MongoDB	Redis
CREATE	−86%	+23%	+21%	−4%
READ	−97%	−16%	−46%	−2%
UPDATE	−88%	+7%	−57%	0%
DELETE	−88%	−25%	−2%	−4%

Kluczowe obserwacje:

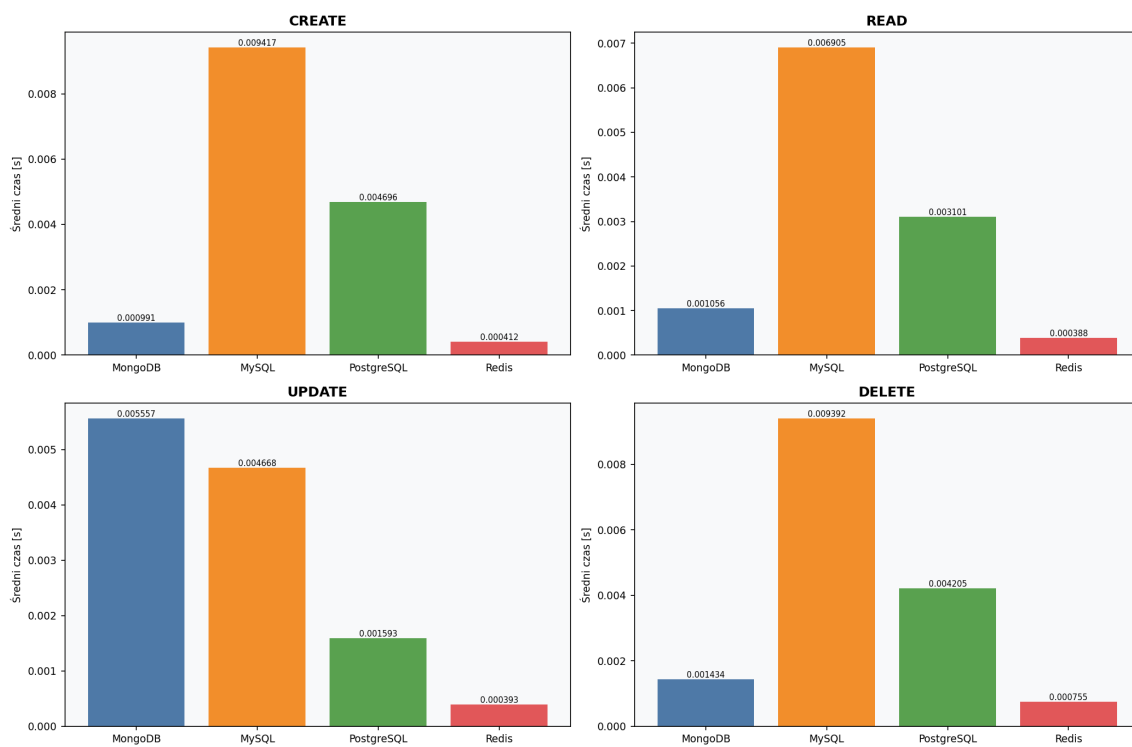
- PostgreSQL zyskuje najbardziej na indeksach – operacje READ przyspieszyły o **97%** (z 16.61 ms do 0.57 ms dla 500k).
- W PostgreSQL nawet operacje CREATE przyspieszyły (mniej czasu na walidację FK z indeksem).
- MongoDB: READ przyspieszył o 46%, UPDATE o 57% – indeksy na zagnieżdżonych polach dokumentu działają efektywnie.
- Redis jest niezmiennie szybki niezależnie od indeksów ($O(1)$ dla operacji klucz-wartość).
- MySQL wykazuje niejednorodne wyniki – CREATE wolniejsze z indeksami (+23%), co potwierdza koszt utrzymania indeksów.

8 Wizualizacja wyników



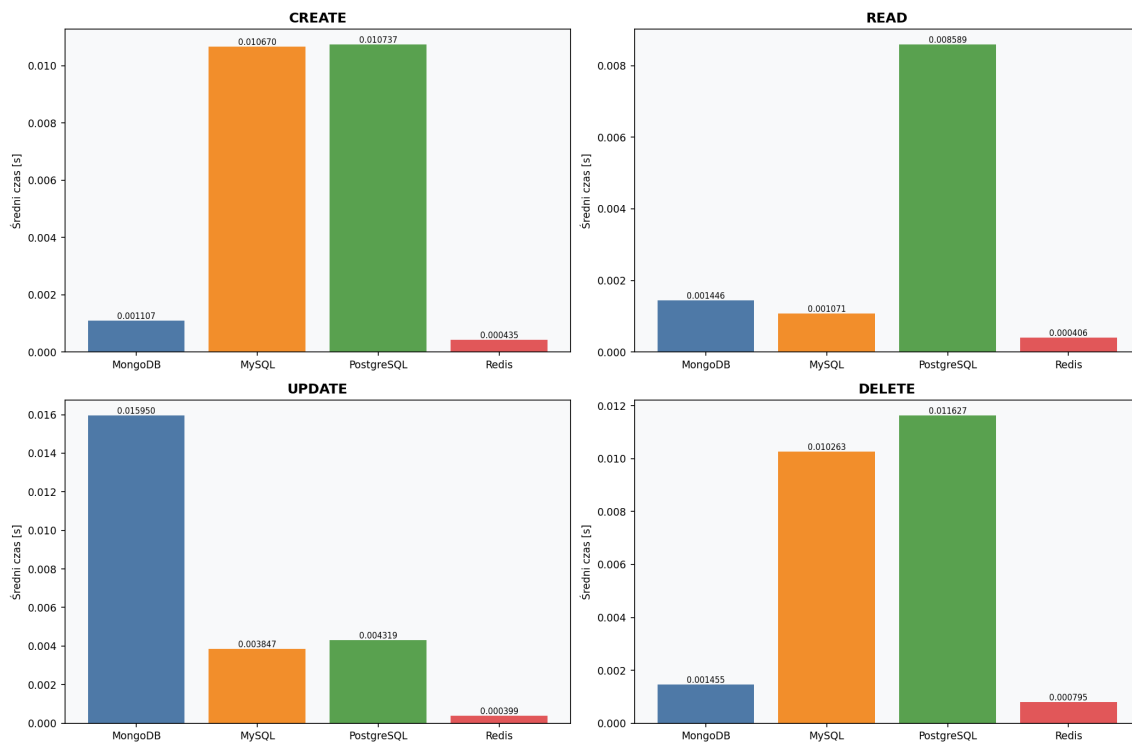
Rysunek 1: Porównanie wydajności CRUD – skala 10 000 rekordów

Porównanie wydajności CRUD – skala 100,000



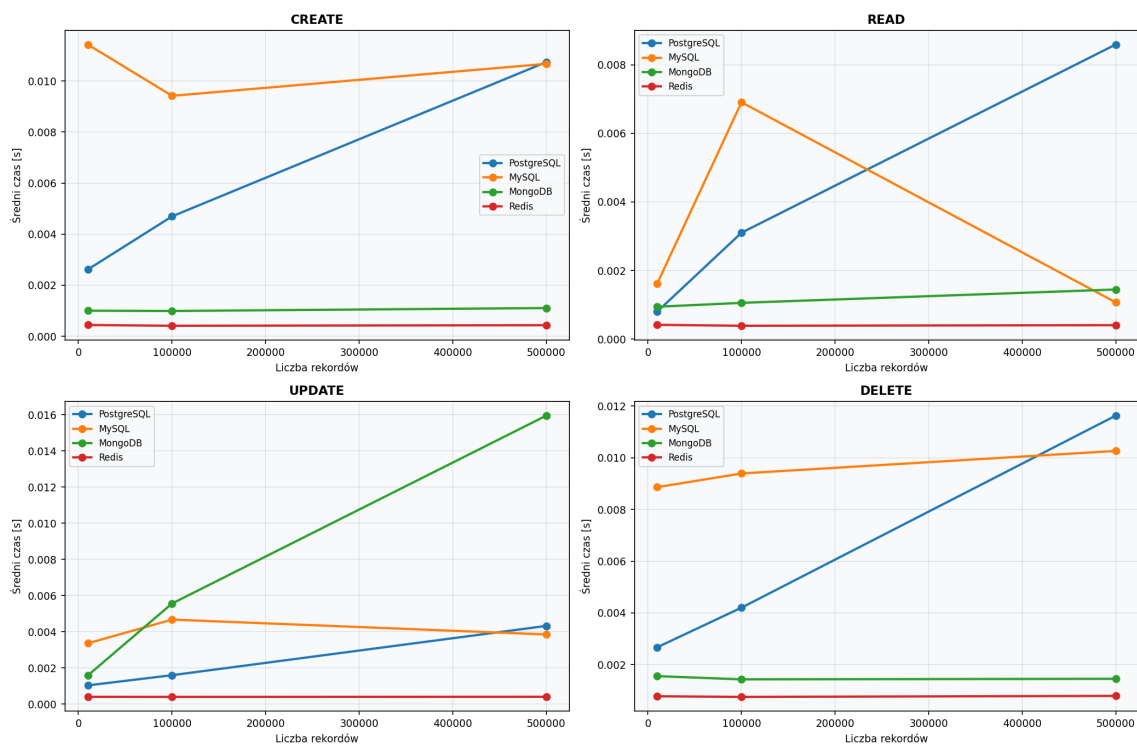
Rysunek 2: Porównanie wydajności CRUD – skala 100 000 rekordów

Porównanie wydajności CRUD – skala 500,000

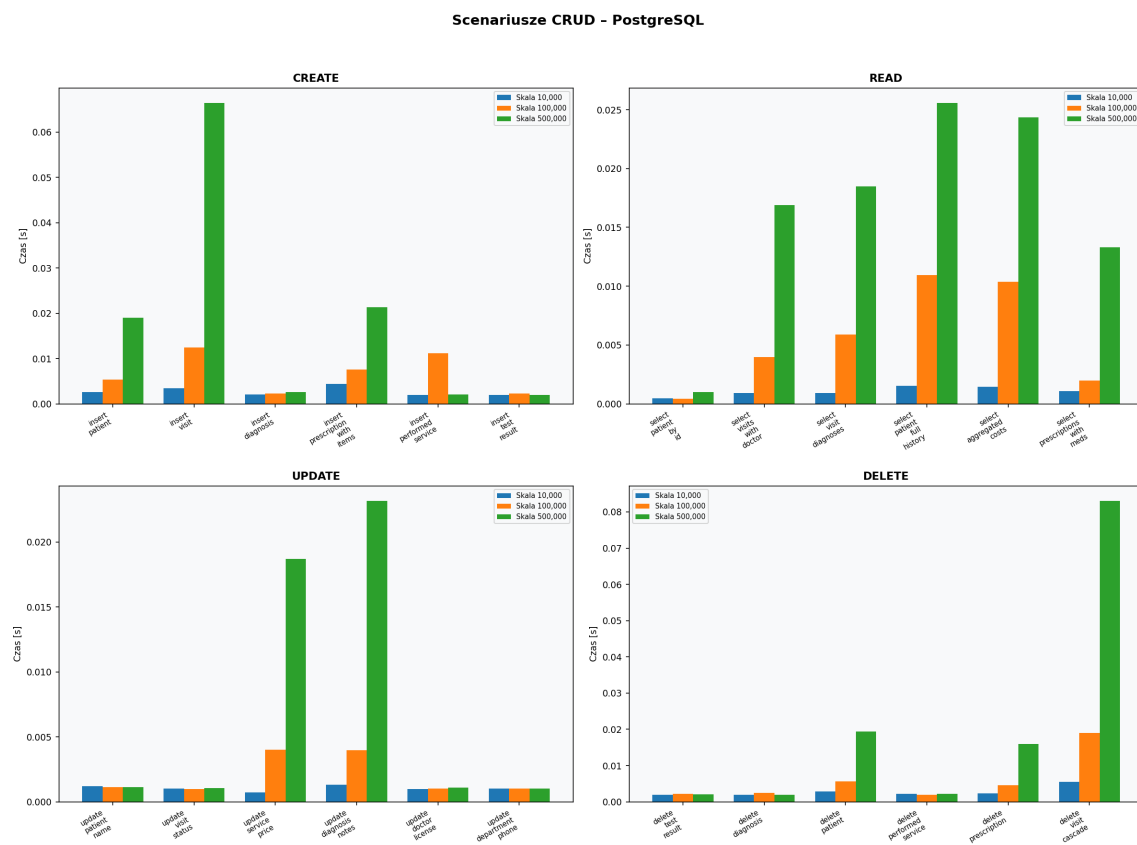


Rysunek 3: Porównanie wydajności CRUD – skala 500 000 rekordów

Skalowalność – wpływ rozmiaru danych na wydajność

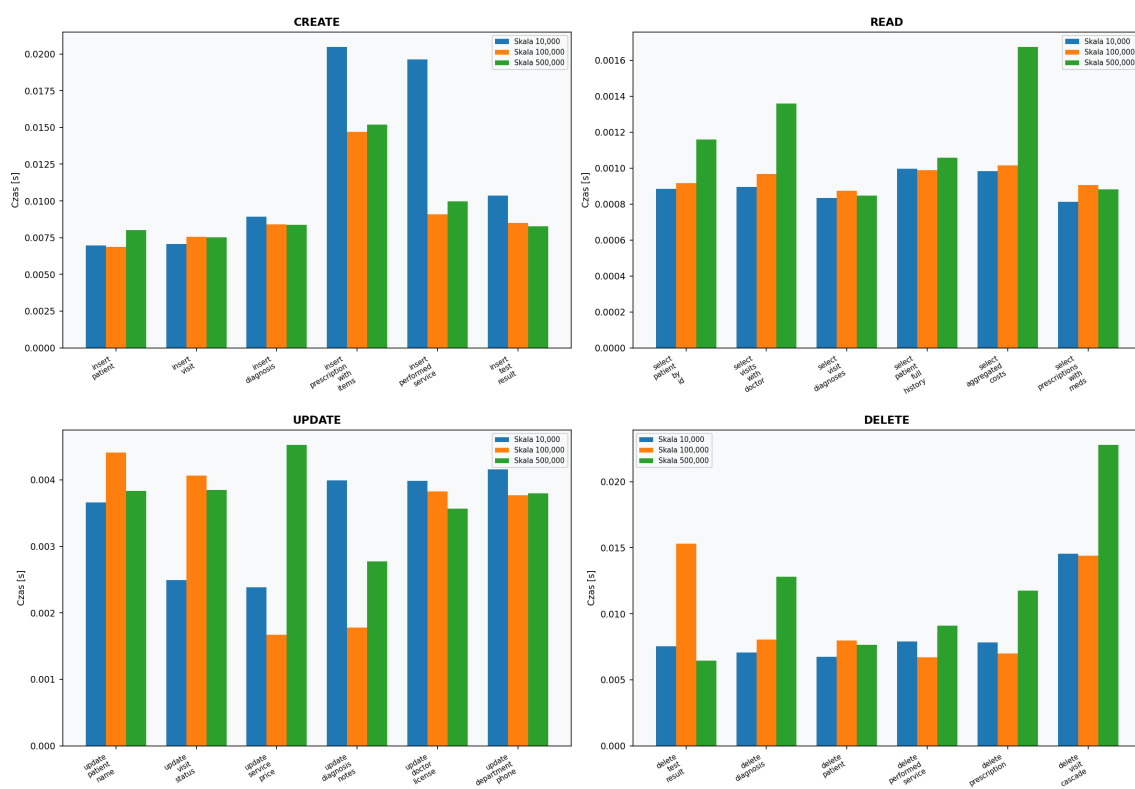


Rysunek 4: Skalowalność – wpływ rozmiaru danych na średni czas operacji CRUD

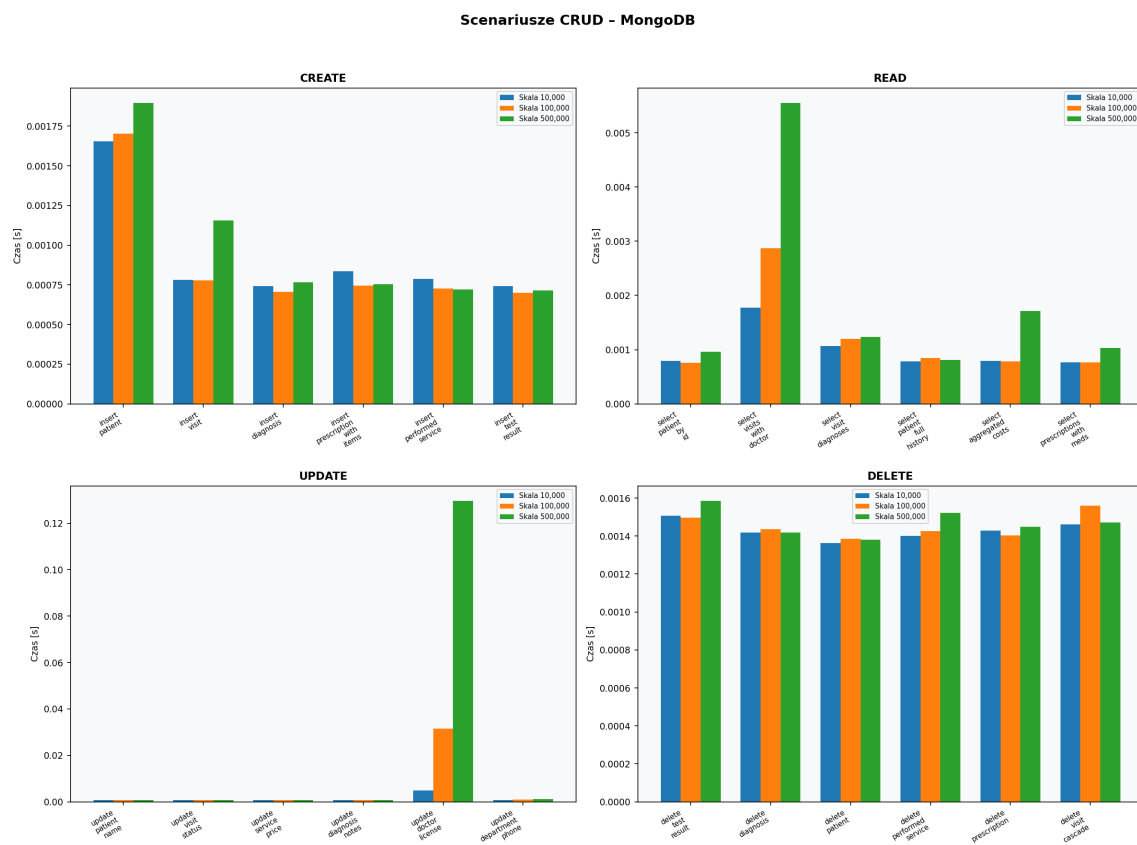


Rysunek 5: Szczegółowe wyniki scenariuszy – PostgreSQL

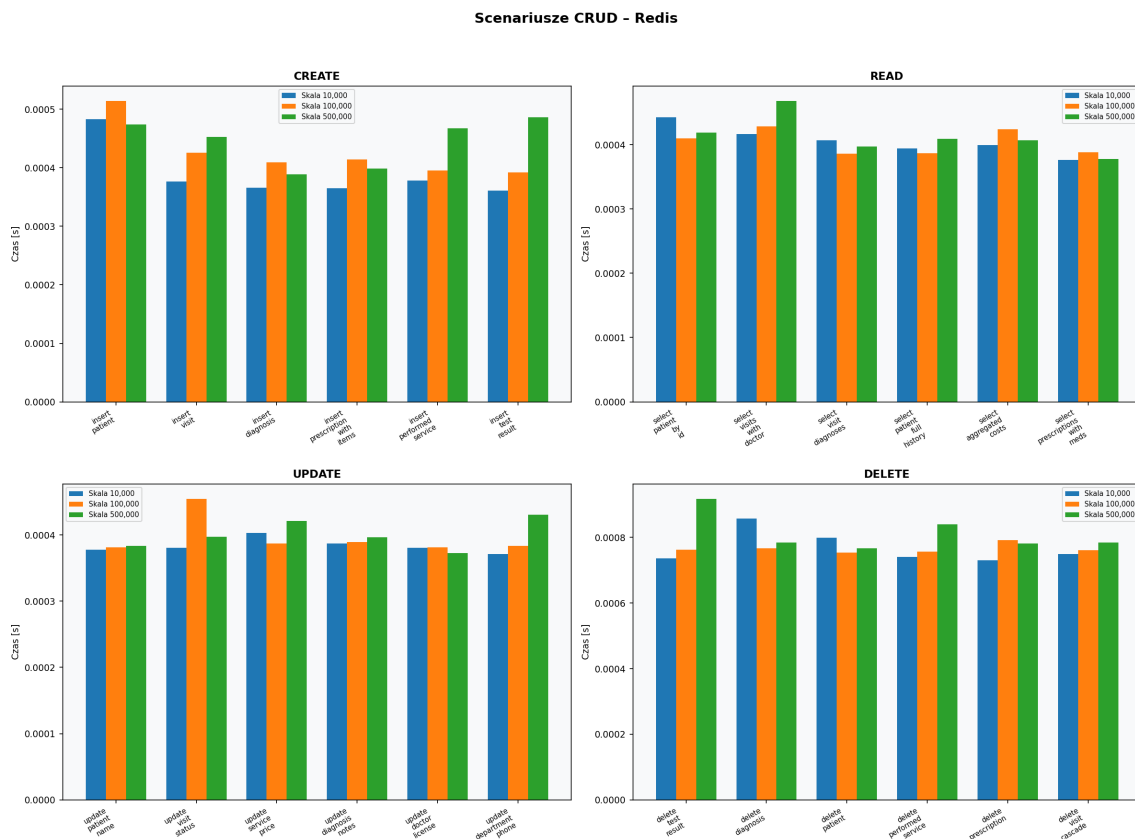
Scenariusze CRUD – MySQL



Rysunek 6: Szczegółowe wyniki scenariuszy – MySQL



Rysunek 7: Szczegółowe wyniki scenariuszy – MongoDB



Rysunek 8: Szczegółowe wyniki scenariuszy – Redis

9 Analiza indeksów

9.1 Zastosowane indeksy

W systemach relacyjnych utworzono 16 indeksów B-tree:

Listing 2: Indeksy w PostgreSQL i MySQL

```
CREATE INDEX idx_patients_last_name ON patients(last_name);
CREATE INDEX idx_patients_national_id ON patients(national_id);
CREATE INDEX idx_patients_gender ON patients(gender);
CREATE INDEX idx_visits_patient_id ON visits(patient_id);
CREATE INDEX idx_visits_doctor_id ON visits(doctor_id);
CREATE INDEX idx_visits_status ON visits(status);
CREATE INDEX idx_visits_date ON visits(visit_date);
CREATE INDEX idx_performed_services_visit_id ON performed_services(
    visit_id);
CREATE INDEX idx_diagnoses_visit_id ON diagnoses(visit_id);
CREATE INDEX idx_diagnoses_disease_id ON diagnoses(disease_id);
CREATE INDEX idx_prescriptions_visit_id ON prescriptions(visit_id);
CREATE INDEX idx_prescription_items_prescription_id
    ON prescription_items(prescription_id);
CREATE INDEX idx_test_results_visit_id ON test_results(visit_id);
CREATE INDEX idx_test_results_parameter ON test_results(parameter_name);
CREATE INDEX idx_doctors_department ON doctors(department_id);
CREATE INDEX idx_doctors_specialization ON doctors(specialization_id);
```

W MongoDB utworzono indeksy na kolekcji patients:

Listing 3: Indeksy w MongoDB

```

db.patients.createIndex({ "last_name": 1 })
db.patients.createIndex({ "national_id": 1 })
db.patients.createIndex({ "gender": 1 })
db.patients.createIndex({ "visits.status": 1 })
db.patients.createIndex({ "visits.doctor_id": 1 })
db.patients.createIndex({ "visits.visit_date": 1 })
db.patients.createIndex({ "visits.diagnoses.disease_id": 1 })

```

10 Analiza planów zapytań (EXPLAIN)

10.1 PostgreSQL – EXPLAIN ANALYZE

Zapytanie R2 – JOIN wizyt z lekarzami (skala 500 000, z indeksami):

Listing 4: Plan zapytania PostgreSQL – JOIN wizyt z lekarzami

```

Hash Join (cost=17.51..59.87 rows=11 width=33)
  (actual time=0.128..0.140 rows=7 loops=1)
  Hash Cond: (v.doctor_id = d.id)
  -> Bitmap Heap Scan on visits v
    (cost=4.51..46.84 rows=11 width=22)
    (actual time=0.049..0.060 rows=7 loops=1)
    Recheck Cond: (patient_id = 1)
    -> Bitmap Index Scan on idx_visits_patient_id
      (cost=0.00..4.50 rows=11 width=0)
      (actual time=0.045..0.045 rows=7 loops=1)
  -> Hash (cost=8.00..8.00 rows=400 width=19)
    (actual time=0.073..0.074 rows=400 loops=1)
Planning Time: 0.180 ms
Execution Time: 0.159 ms

```

Optymalizator PostgreSQL wykorzystuje **Bitmap Index Scan** na indeksie `idx_visits_patient_id`, a następnie **Hash Join** z tabelą `doctors`. Całkowity czas wykonania to zaledwie **0.159 ms** przy 500 000 wizytach.

Zapytanie z agregacją – SUM kosztów usług:

Listing 5: Plan zapytania PostgreSQL – agregacja z GROUP BY

```

GroupAggregate (cost=140.28..140.50 rows=11 width=36)
  -> Sort (cost=140.28..140.31 rows=11 width=10)
    -> Nested Loop (cost=4.93..140.09 rows=11 width=10)
      -> Bitmap Heap Scan on visits v
        -> Bitmap Index Scan on idx_visits_patient_id
      -> Index Scan using idx_performed_services_visit_id
        on performed_services ps
Planning Time: 0.146 ms
Execution Time: 0.100 ms

```

PostgreSQL stosuje **Nested Loop** z dwoma skanami indeksowymi, co daje czas wykonania **0.100 ms**.

10.2 MySQL – EXPLAIN FORMAT=JSON

MySQL wykorzystuje ten sam indeks `idx_visits_patient_id` z typem dostępu `ref` (przeszukanie indeksu). Koszt zapytania wynosi **4.90** (w jednostkach wewnętrznych My-)

SQL). Plan potwierdza użycie `nested_loop` z dostępem `eq_ref` do tabeli `doctors` przez klucz główny.

10.3 MongoDB – explain executionStats

Listing 6: Plan zapytania MongoDB – find po `_id`

```
executionSuccess: True
nReturned: 1
executionTimeMillis: 0
totalKeysExamined: 1
totalDocsExamined: 1
stage: IDHACK
```

Zapytanie po `_id` wykorzystuje optymalizację `IDHACK` – bezpośredni dostęp do dokumentu w czasie < 1 ms.

Zapytanie po zagnieżdżonym polu `visits.status = 'completed'` z indeksem:

Listing 7: Plan zapytania MongoDB – find po statusie wizyty

```
executionSuccess: True
nReturned: 45903
executionTimeMillis: 42
totalKeysExamined: 45903
totalDocsExamined: 45903
stage: FETCH
```

Indeks na `visits.status` pozwala na efektywne przefiltrowanie – **42 ms** dla zwrócenia 45 903 dokumentów ze zbioru 50 000.

11 Weryfikacja hipotezy badawczej

H1: *Zastosowanie indeksów znacząco poprawia wydajność operacji `SELECT` kosztem spadku wydajności operacji `INSERT` i `UPDATE`.*

11.1 Analiza wyników

Tabela 11: Weryfikacja hipotezy – PostgreSQL, skala 500 000

Operacja	Bez indeksów [ms]	Z indeksami [ms]	Zmiana
CREATE (INSERT)	18.90	2.58	–86% (przyspieszenie)
READ (SELECT)	16.61	0.57	– 97% (przyspieszenie)
UPDATE	7.69	0.95	–88% (przyspieszenie)
DELETE	20.78	2.48	–88% (przyspieszenie)

11.2 Wnioski dotyczące hipotezy

Hipoteza H1 została **częściowo potwierdzona**:

1. **Potwierdzenie:** Operacje READ (SELECT) zyskały najwyższe przyspieszenie – **97%** w PostgreSQL dla skali 500 000. Indeksy eliminują konieczność sekwencyjnego skanowania (Seq Scan) na rzecz skanów indeksowych (Index Scan, Bitmap Index Scan).
2. **Odrzucenie części hipotezy:** Wbrew oczekiwaniom, operacje INSERT i UPDATE w PostgreSQL **również przyspieszyły** po zastosowaniu indeksów. Wynika to z faktu, że scenariusze INSERT obejmują wstawianie rekordów z kluczami obcymi – indeksy na kolumnach referencyjnych przyspieszają walidację FK. Ponadto operacje UPDATE korzystają z indeksów do szybszego lokalizowania aktualizowanego wiersza.
3. **MySQL potwierdza hipotezę:** W MySQL operacje CREATE są wolniejsze z indeksami (+23%), co jest zgodne z klasycznym kompromisem indeks vs. zapis. Każdy INSERT wymaga aktualizacji 16 dodatkowych struktur indeksowych.
4. **Redis:** Niezależny od indeksów – operacje $O(1)$ na strukturach klucz-wartość.

12 Rozszerzona analiza wyników

12.1 Skalowalność

Na podstawie wykresu skalowalności (Rysunek 4) można zaobserwować:

- **Redis:** Stały czas operacji niezależnie od rozmiaru danych (~ 0.4 ms). Potwierdza złożoność $O(1)$ operacji klucz-wartość.
- **MongoDB:** Liniowy wzrost czasu READ i UPDATE wraz ze wzrostem danych (bez indeksów). Z indeksami wzrost jest minimalny.
- **PostgreSQL:** Bez indeksów – gwałtowny wzrost czasu READ (od 1.07 ms do 16.61 ms dla 50x większych danych). Z indeksami – prawie stały czas.
- **MySQL:** Relatywnie stabilne czasy CREATE/UPDATE niezależnie od skali (buforowanie InnoDB).

12.2 Porównanie modeli danych

Model relacyjny (PostgreSQL, MySQL):

- Wymaga JOINów, ale z indeksami osiąga doskonałą wydajność.
- Silne gwarancje spójności (ACID, klucze obce).
- Lepszy dla złożonych zapytań analitycznych.

Model dokumentowy (MongoDB):

- Eliminuje JOINy dzięki zagnieżdżeniom.
- Szybsze odczyty pojedynczych dokumentów.
- Wolniejsze UPDATE na zagnieżdżonych tablicach (szczególnie update_many).

Model klucz-wartość (Redis):

- Najszybszy dla prostych operacji CRUD.
- Brak możliwości złożonych zapytań.
- Idealny jako warstwa cache.

12.3 Dane półustrukturalne (JSON)

MongoDB przechowuje dane w formacie BSON (Binary JSON), co stanowi wykorzystanie danych półustrukturalnych w projekcie. Dokumenty pacjentów zawierają zagnieżdżone tablice wizyt, diagnoz, recept – bez sztywnego schematu. PostgreSQL również obsługuje JSON/JSONB, jednak w tym projekcie skupiono się na modelu relacyjnym z normalizacją.

12.4 Automatyzacja testów

Projekt realizuje pełną automatyzację:

- Jeden skrypt `run_all.py` uruchamia kompletny pipeline benchmarkowy.
- Automatyczne generowanie danych, wstawianie, testy, raporty i wykresy.
- Parametryzowalne skale: `python run_all.py -scales 10000 100000 500000`.
- Wyniki zapisywane w formacie CSV, wykresy w PNG, raport EXPLAIN w TXT.

13 Podsumowanie i wnioski

1. **Redis** jest bezkonkurencyjnie najszybszy dla prostych operacji CRUD (~ 0.4 ms), ale nie nadaje się do złożonych zapytań relacyjnych.
2. **PostgreSQL z indeksami** oferuje najlepszy stosunek wydajności do funkcjonalności – operacje READ w 0.57 ms nawet przy 500 000 wizytach, zachowując pełne ACID i integralność referencyjną.
3. **MongoDB** sprawdza się doskonale dla odczytów pojedynczych dokumentów i systemów o elastycznym schemacie, ale operacje UPDATE na zagnieżdżonych strukturach są kosztowne przy dużych zbiorach.
4. **MySQL** wykazuje stabilną wydajność niezależnie od skali, ale jest wolniejszy od PostgreSQL w operacjach z wieloma JOINami.
5. **Indeksy są kluczowe** dla wydajności systemów relacyjnych – przyspieszają READ o nawet 97%, a w PostgreSQL przyspieszają również operacje zapisu (dzięki szybszej walidacji FK).
6. **Wybór SZBD** powinien być podyktowany charakterem aplikacji:
 - Złożone zapytania analityczne → PostgreSQL
 - Proste aplikacje webowe → MySQL
 - Elastyczny schemat, agregacje → MongoDB
 - Cache, sesje, kolejki → Redis